EXPERIMENT 07

Integrating a Zoho Creator Application with Another Zoho Service Using Connections and API

Patient Appointment Booking Form — Zoho Calendar Integration Use Case

KAVIYA
241401038

1. Aim

To integrate the Appointment_Booking_Form application built in Zoho Creator with Zoho Calendar — by creating an authorised connection and calling the Zoho Calendar REST API through a Deluge script in a workflow — so that a calendar event is created automatically the moment a patient submits the booking form with consent.

This removes the need for front-desk staff to enter every appointment into the calendar by hand, keeping the hospital schedule consistent with the data captured in the application.

2. Procedure

Step 1: Open Microservices — From the Develop section of the Zoho Creator home page, **Microservices** was opened and **Create New** was selected.

Step 2: Choose Connections — In the Create New Microservice dialog, **Connections** (connect apps with third-party services) was chosen using Pick & Create, and **Zoho Calendar** was selected as the service.

Step 3: Configure the connection — The connection was named **zoho_calendar_connection**, with Quick connect (Zoho credential) mode and the required Calendar permissions, and submitted with **Create And Authorize**.

Step 4: Authorise for all environments — In the **Zoho Calendar – Link Account** dialog, Development, Staging and Production were selected and **Authorize** was clicked; the consent screen was accepted.

Step 5: Enable the connection for the app — The saved connection shows Service Name **Zoho Calendar**, link name **zoho_calendar_connection**, permissions Create Calendar, Create Event and Get all Calendar, and status Enabled. **Connection Access** was switched On for Appointment_Booking_Form.

Step 6: Create the workflow — A workflow named **Sync_Appointment_To_Zoho_Calendar** was created on Appointment_Booking_Form with Record Event **Created** and Form Event **Successful form submission**.

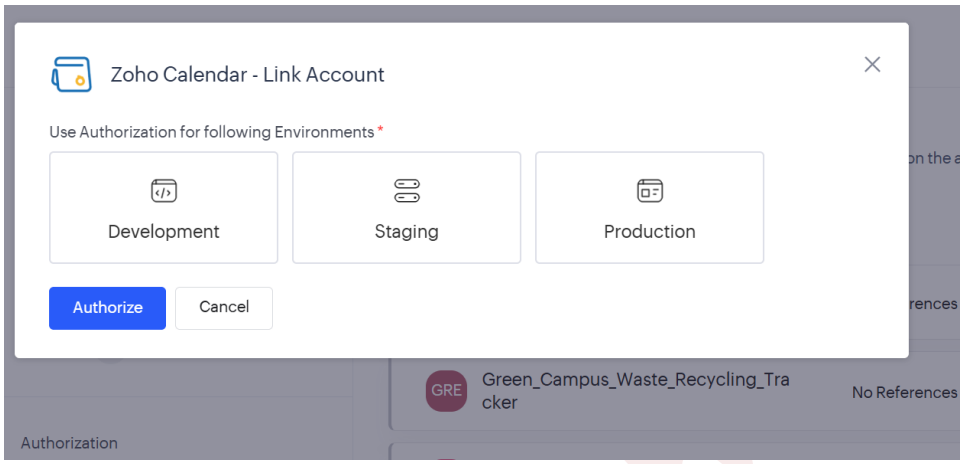
Step 7: Write the Deluge script — If consent is given, the script (1) calls **GET /api/v1/calendars** through the connection to read the calendar UID, (2) builds the start and end values (yyyyMMddTHHmmss+0530, one-hour slot) from the preferred date and time, (3) builds the event map with title “Patient Appointment – <patient name>” and the department and age as description, and (4) calls **POST /api/v1/calendars/{uid}/events** with **invokeurl**, sending the event as **eventdata**. Each step is logged with **info**.

Step 8: Test and debug — Records were submitted through the live form, the workflow logs were read after each run, errors were fixed, and the final run was verified in Zoho Calendar.

241401038

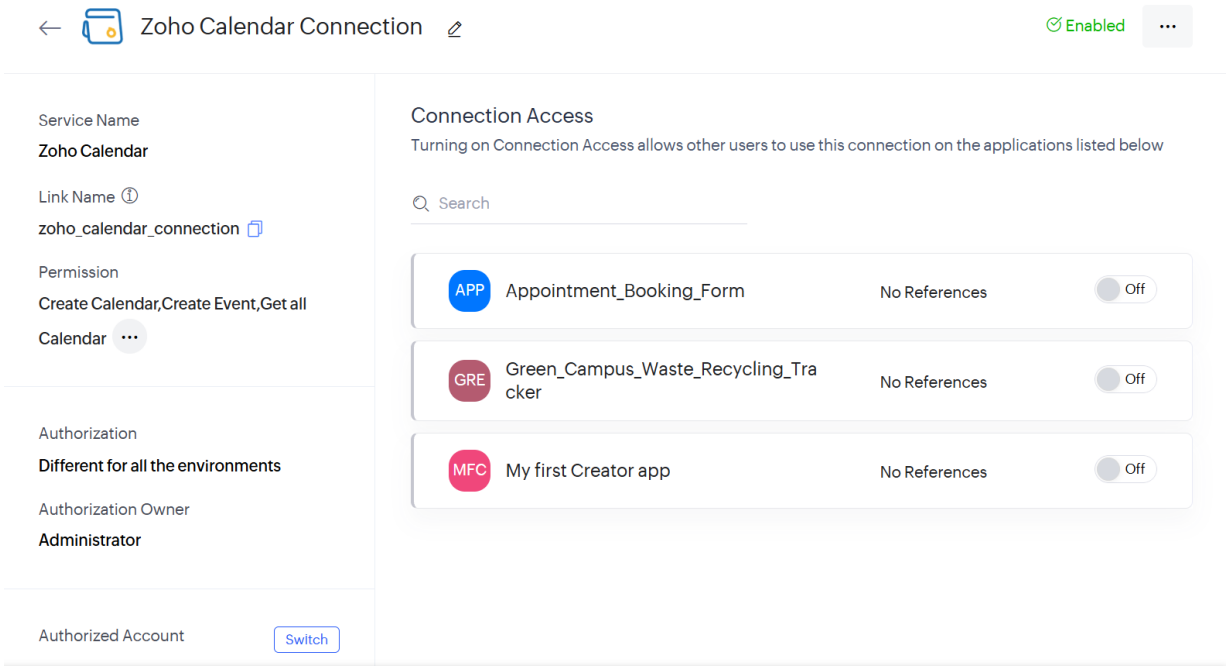
3. Output

3.1 Authorising the Zoho Calendar connection



The Zoho Calendar – Link Account dialog. Authorisation was applied to the Development, Staging and Production environments before clicking Authorize.

3.2 Connection details and access



The saved connection: Service Name Zoho Calendar, link name **zoho_calendar_connection**, permissions Create Calendar, Create Event and Get all Calendar, status Enabled. Connection Access lists the applications that can use it; it was switched On for Appointment_Booking_Form.

3.3 Workflow and Deluge script



The breadcrumb **Appointment_Booking_Form › Created › Successful form submission › Sync_Appointment_To_Zoho_Calendar** with the Deluge script action, executed without condition on every successful new submission.

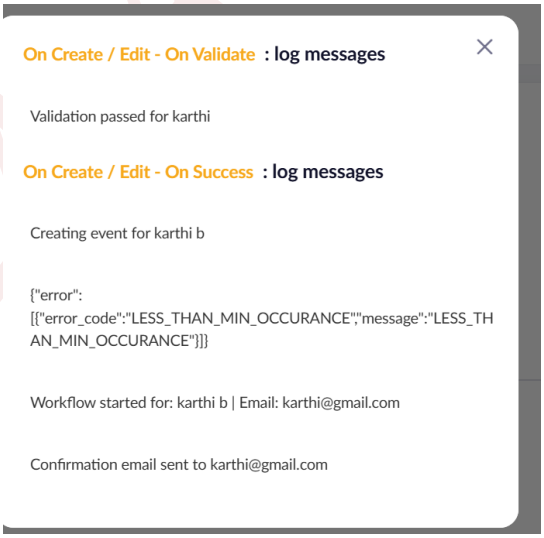
Final Deluge script:

```
if(input.Consent_For_Treatment != null && input.Consent_For_Treatment.size() > 0)
{
    patientName = input.Patient_namePatient_name.first_name + " "
                + input.Patient_namePatient_name.last_name;
    // 1. Get the calendar ID from Zoho Calendar
    calendarId = "";
    calResp = invokeurl
    [
        url :"https://calendar.zoho.in/api/v1/calendars"
        type :GET
        connection:"zoho_calendar_connection"
    ];
    calList = calResp.get("calendars");
    if(calList != null && calList.size() > 0)
    {
        calendarId = calList.get(0).get("uid");
    }
    info "Calendar ID used: " + calendarId;
    // 2. Build start and end time
    datePart = input.Preferred_Appointment_Date.toString("yyyyMMdd");
    timeDigits = "100000";
    if(input.Preferred_Appointment_Time != null)
    {
        timeDigits = input.Preferred_Appointment_Time.toString().replaceAll(":","").trim();
    }
    hh = timeDigits.subString(0,2).toLong();
    rest = timeDigits.subString(2,6);
    endHH = hh + 1;
    if(endHH > 23)
    {
        endHH = 23;
        rest = "5900";
    }
    endHHStr = endHH.toString();
    if(endHH < 10)
    {
```

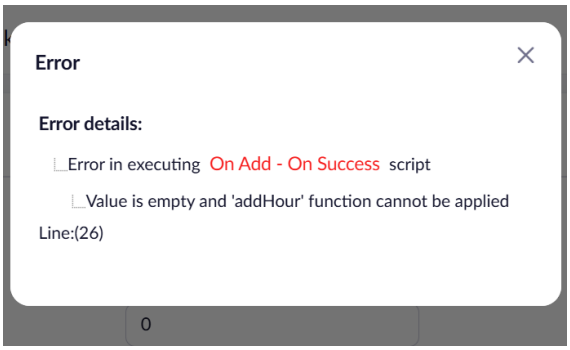
```
        endHHStr = "0" + endHHStr;
    }
    startStr = datePart + "T" + timeDigits + "+0530";
    endStr = datePart + "T" + endHHStr + rest + "+0530";
    info "Start: " + startStr + " | End: " + endStr;
    // 3. Build the event
    dateAndTime = Map();
    dateAndTime.put("timezone","Asia/Calcutta");
    dateAndTime.put("start",startStr);
    dateAndTime.put("end",endStr);
    eventMap = Map();
    eventMap.put("dateandtime",dateAndTime);
    eventMap.put("title","Patient Appointment - " + patientName);
    eventMap.put("description","Department: " + input.Department + " | Age: " + input.Age);
    // 4. Create the event using the Zoho Calendar REST API
    params = Map();
    params.put("eventdata",eventMap.toString());
    response = invokeurl
    [
        url :"https://calendar.zoho.in/api/v1/calendars/" + calendarId + "/events"
        type :POST
        parameters:params
        connection:"zoho_calendar_connection"
    ];
    info response;
}
else
{
    info "Consent not given - calendar event not created";
}
```

3.4 Testing and debugging — errors found and fixed

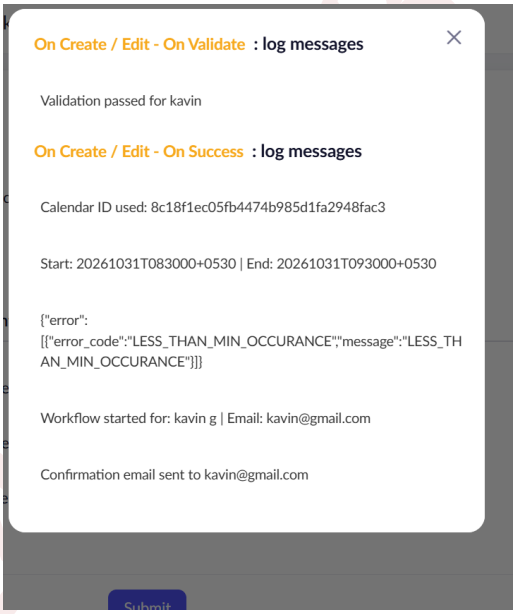
Error 1 — API returned LESS_THAN_MIN_OCCURANCE. The first version used the built-in **zoho.calendar.createEvent** task with a manually copied calendar ID; the log showed the request was rejected because a required value was missing.



Error 2 — "Value is empty and addHour function cannot be applied" (line 26). Converting the date and time into a single date-time value returned empty. **Fix:** the start and end strings were built directly from the date (yyyyMMdd) and the time digits instead of a date-time conversion.



Error 3 — LESS_THAN_MIN_OCCURANCE even with the correct calendar ID and times. The added **info** logs proved the calendar UID (fetched via the API) and the start/end values were now correct, so the problem was the createEvent task itself. **Fix:** the event was created with a direct REST call — **POST /api/v1/calendars/{uid}/events** via **invokeurl** with the event JSON in the **eventdata** parameter, using the same connection.

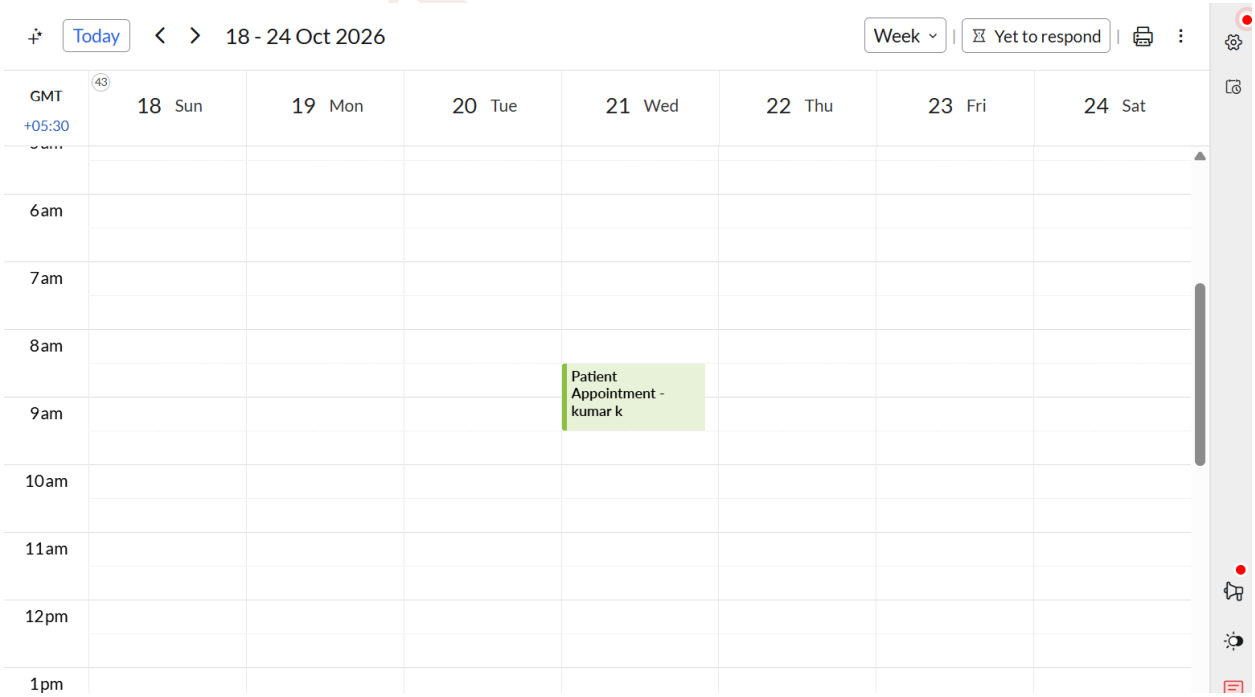


3.5 API response — event created



The workflow log after the fix: the calendar UID and the start/end values, followed by the Zoho Calendar API response **{"events":[...]}** containing the new event — title “Patient Appointment – kumar k”, description “Department: Dental | Age: 20”, start 20261021T083000+0530 and end 20261021T093000+0530.

3.6 Output — event in Zoho Calendar



Zoho Calendar (week of 18–24 Oct 2026) showing the event **Patient Appointment – kumar k** on Wednesday 21 Oct from 8:30 to 9:30 am, created automatically from the submitted Creator record.

4. Final Result

A connection to Zoho Calendar (**zoho_calendar_connection**) was created, authorised and enabled for the Appointment_Booking_Form application, and a workflow on the booking form was built that uses it through Deluge and the Zoho Calendar REST API.

On every successful submission with consent, the workflow reads the calendar UID through the API, builds the appointment slot from the preferred date and time (Asia/Calcutta, +0530), and creates the event "Patient Appointment – <patient name>" in Zoho Calendar. Three errors found during testing were debugged using **info** logs and fixed.

The exercise met its objective: a Zoho Creator application was integrated with another Zoho service using a connection and API calls from Deluge, so appointment data now flows automatically into the calendar without manual entry.

241401038